Machine Learning Operations (AIMLCZG523)

Assignment 01

Heart Disease Risk Prediction

End-to-end ML model development, CI/CD and production deployment

Student	C V R Karthik
BITS ID	2025AE05851
Code repository	https://github.com/<your-account>/heart-disease-mlops
Pipeline video	<link to pipeline video recording>
Champion model	Logistic Regression - CV ROC-AUC 0.888, hold-out ROC-AUC 0.930
Stack	scikit-learn, MLflow, FastAPI, Docker, Kubernetes (Minikube), GitHub Actions, Prometheus, Grafana

Contents

Contents 2
1. Introduction and architecture 3
Key design decisions 3
2. Setup and installation 3
3. Data acquisition and exploratory analysis 4
3.1 Acquisition and cleaning 4
3.2 Findings 4
4. Feature engineering and model development 6
4.1 Feature pipeline 6
4.2 Models and tuning 6
4.3 Results 7
4.4 Model selection 7
5. Experiment tracking (MLflow) 8
6. Model packaging and reproducibility 9
7. CI/CD pipeline and automated testing 10
7.1 Unit tests 10
7.2 GitHub Actions workflow 10
8. Model containerisation 11
9. Production deployment (Kubernetes) 11
10. Monitoring and logging 12
10.1 Logging 12
10.2 Metrics, dashboard and alerts 12
11. Conclusion and deliverables 13
Possible extensions 13

1. Introduction and architecture

The goal is a **reproducible, monitored, cloud-ready API** that estimates the risk of heart disease from routine clinical measurements. The work follows the full MLOps lifecycle: data acquisition and EDA, feature engineering and model tuning, experiment tracking, packaging, automated testing and CI/CD, containerisation, Kubernetes deployment and runtime monitoring.

Figure 1 shows the system. Everything is driven from one Python package (`src/heart`), so the same code runs in notebooks, in the CLI, in CI, and inside the serving container.

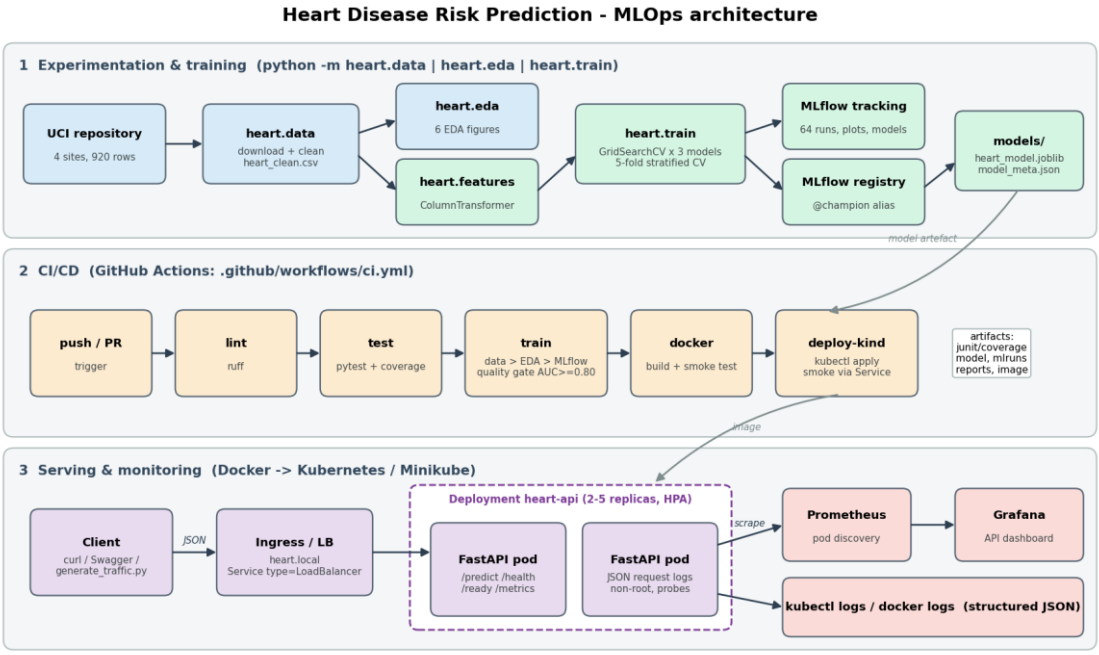


Figure 1. *End-to-end architecture: training lane, CI/CD lane and serving/monitoring lane.*

Key design decisions

- **All four UCI sites (920 patients)** are used rather than the usual 303-row Cleveland subset. This gives three times more data and makes the pipeline handle real missing values.
- **Preprocessing is part of the persisted model** (one sklearn `Pipeline`), so training and serving can never diverge.
- **Model selection uses cross-validation only**; the 20 % hold-out set is used just for reporting.
- **Missingness is modelled explicitly** (indicator flags plus a 'missing' category level), and the API accepts patients with unmeasured fields.
- **Observability is built in**: structured JSON request logs, Prometheus metrics (including a prediction-mix drift signal), a provisioned Grafana dashboard and alert rules.

2. Setup and installation

Prerequisites: Python 3.10-3.12, Docker, Git and internet access. `kubectl` and `minikube` are installed automatically into `~/.local/bin` if missing. The whole pipeline runs with one command:

```
git clone https://github.com/<your-account>/heart-disease-mlops
cd heart-disease-mlops
bash scripts/lab_run.sh all        # setup data eda train test nb mlflow docker k8s traffic
```

Each stage can also be run on its own, and each writes its console output to `logs/<stage>.log`:

Stage	Command / effect
setup	`python -m venv .venv` and `pip install -r requirements.txt` (pinned versions)
data / eda / train	`python -m heart.data`, `python -m heart.eda`, `python -m heart.train`
test	`ruff check .` and `pytest --cov=heart`

Stage	Command / effect
mlflow	`mlflow ui --backend-store-uri sqlite:///mlflow.db` on port 5000
docker	build `heart-disease-api:1.0.0`, run it, 9-check smoke test, show container logs
k8s	minikube start, `minikube image load`, `scripts/deploy_k8s.sh`, smoke test through the Service
traffic / compose	sample load for dashboards / Docker-Compose alternative (API, Prometheus, Grafana)

Windows without Docker: create the venv, `pip install -r requirements.txt`, run the three `python -m heart.*` modules, then `uvicorn heart.serving:app`.

3. Data acquisition and exploratory analysis

3.1 Acquisition and cleaning

`heart.data.download()` fetches `processed.{cleveland,hungarian,switzerland,va}.data` from the UCI repository and caches them in `data/raw/`. `heart.data.clean()` then applies the following rules (unit-tested in `tests/test_data.py`):

Issue found in raw data	Treatment
`?` used as missing marker	parsed as NaN
Diagnosis `num` is a 0-4 severity	binary target = `num > 0` (presence of disease)
`chol = 0` in 172 rows (all 123 Swiss rows + 49 VA), `trestbps = 0` once	physiologically impossible, so set to NaN
Codes outside the data dictionary	set to NaN (e.g. thal must be 3, 6 or 7)
2 exact duplicate rows	dropped, leaving 918 rows (55.3 % positive)

Imputation is intentionally **not** done during cleaning. It is part of the model pipeline (Section 4), so identical logic is applied to API requests.

3.2 Findings

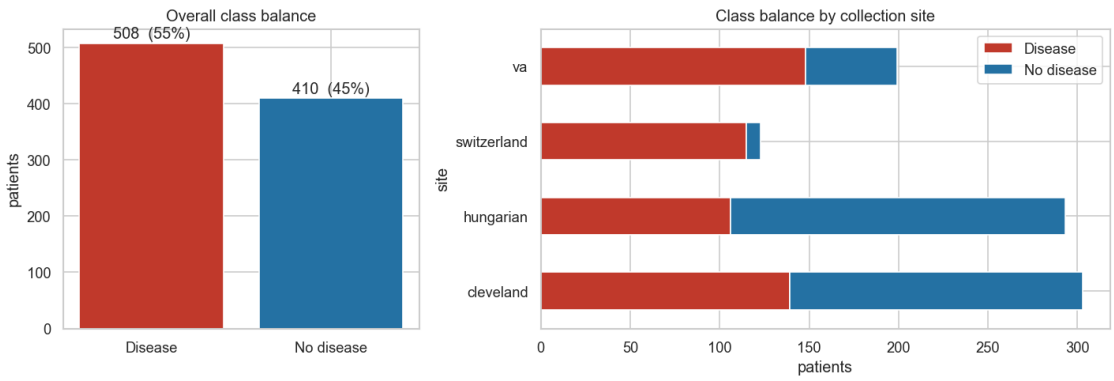


Figure 2. *Class balance overall and per collection site.*



Figure 3. *Missing values (%) per feature and site - missingness is site-driven.*

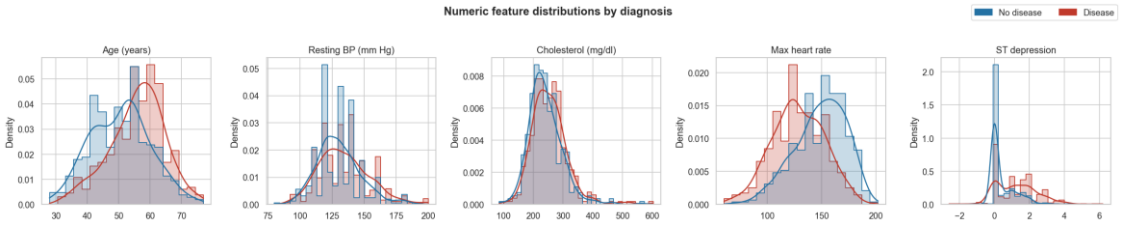


Figure 4. *Distributions of numeric features by diagnosis.*

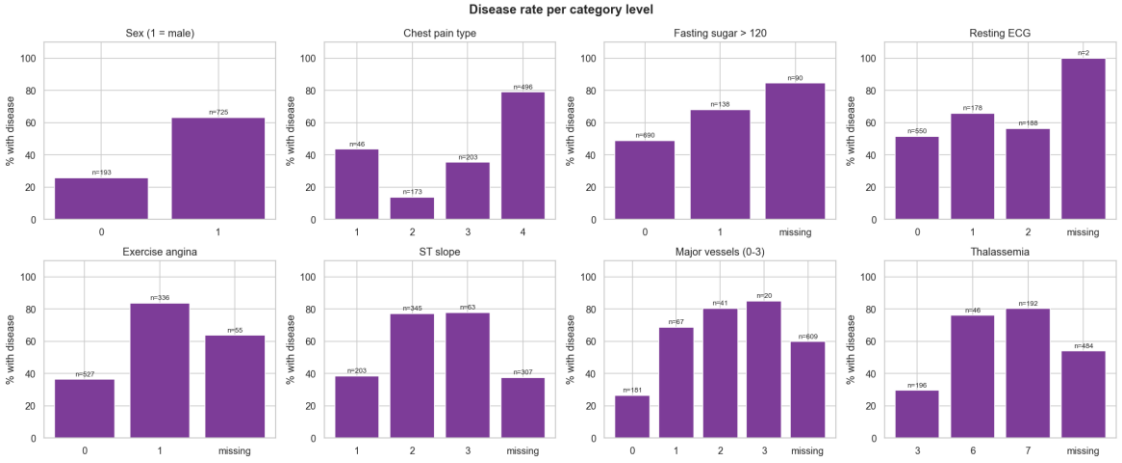


Figure 5. *Disease rate per category level (n shown above each bar).*

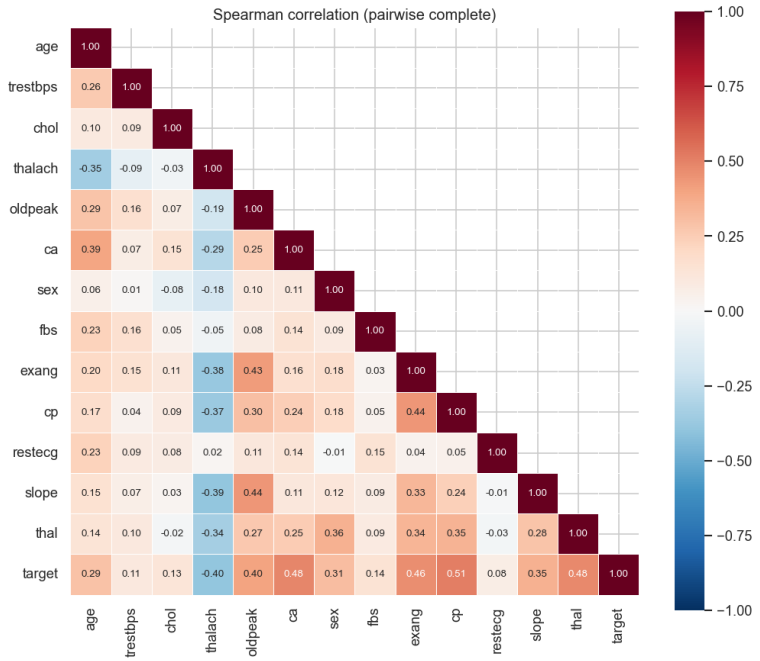


Figure 6. *Spearman correlation heat-map (pairwise complete observations).*

Observation	Consequence for modelling
Balanced overall (55/45) but prevalence ranges from 36 % (Hungary) to 93 % (Switzerland)	stratified splits; `site` excluded from features because it would leak centre prevalence
`ca` 66 %, `thal` 53 %, `slope` 33 % missing, mostly outside Cleveland	median/constant imputation plus missing-indicator features
Asymptomatic chest pain, exercise angina, ST depression, flat ST slope, `ca` > 0 and reversible thal defect are the strongest risk factors; max heart rate is protective	relationships are largely monotone, so linear models are competitive
`oldpeak` is skewed; `chol` has outliers	standard scaling; tree models are robust to the rest

4. Feature engineering and model development

4.1 Feature pipeline

`heart.features.build_preprocessor()` returns one `ColumnTransformer` that turns the 13 inputs into 31 model features:

Group	Columns	Steps
Numeric	age, trestbps, chol, thalach, oldpeak, ca	median impute + missing indicator, then StandardScaler
Binary	sex, fbs, exang	most-frequent impute
Categorical	cp, restecg, slope, thal	missing becomes level -1, then one-hot with a fixed category list (unknown codes map to all-zeros)

An ablation in notebook `02_model_training` (logistic regression, 5-fold CV) confirms the design: ROC-AUC **0.880** without indicators, **0.896** with indicators, and **0.899** with indicators plus a 'missing' category level.

4.2 Models and tuning

Three model families are tuned with `GridSearchCV` (5-fold stratified CV, `refit='roc_auc'`; accuracy, precision, recall and F1 are also scored). ROC-AUC is the selection metric because it does not depend on the decision threshold and is robust to the prevalence shift between sites. A majority-class baseline scores 0.500 ROC-AUC and 0.553 accuracy.

Model	Grid searched	Candidates
Logistic regression (liblinear)	C {0.01, 0.1, 0.5, 1, 5} x penalty {l1, l2} x class_weight {none, balanced}	20
Random forest	n_estimators {200, 400} x max_depth {none, 5, 8} x min_samples_leaf {1, 4} x max_features {sqrt, 0.5}	24
Hist gradient boosting	learning_rate {0.03, 0.1} x max_depth {3, none} x max_iter {100, 250} x l2 {0, 1}	16

4.3 Results

Model	CV ROC-AUC	CV acc.	CV prec.	CV recall	Test ROC-AUC	Test acc.	Test recall
Logistic Regression *	0.888 ± 0.050	0.820	0.828	0.860	0.930	0.842	0.892
Random Forest	0.884 ± 0.052	0.820	0.828	0.857	0.924	0.821	0.863
Hist Gradient Boosting	0.878 ± 0.050	0.809	0.815	0.852	0.919	0.859	0.931

* champion

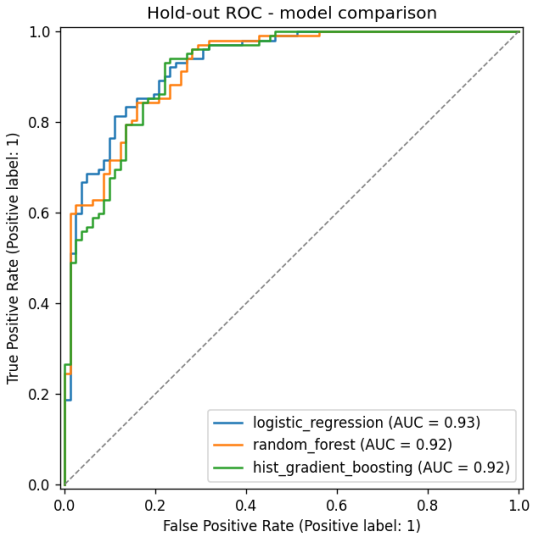


Figure 7. *Hold-out ROC curves of the three tuned models.*

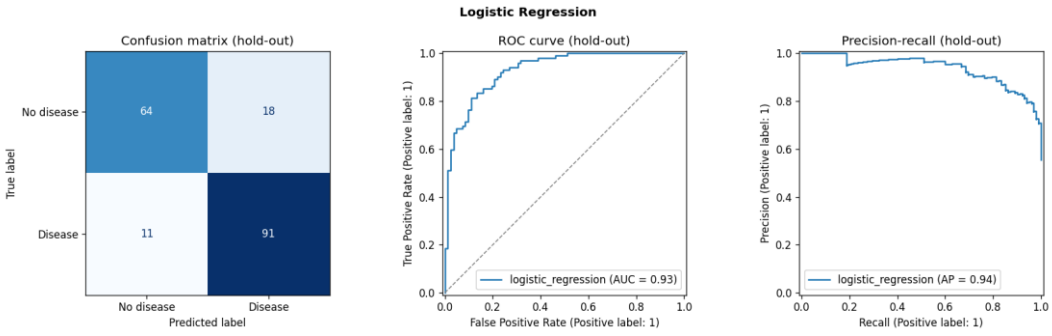


Figure 8. *Champion diagnostics on the hold-out set: confusion matrix, ROC and precision-recall.*

4.4 Model selection

- The three families are within about 0.01 CV ROC-AUC of each other, which is less than one CV standard deviation (about 0.05).
- **Logistic Regression** (penalty=l1, C=0.5) has the best mean CV ROC-AUC (0.888). It is also the simplest and fastest model and gives interpretable coefficients, which matters in a clinical setting, so it was chosen as champion. The L1 penalty zeroes 8 of the 31 features, which acts as built-in feature selection.
- On the untouched hold-out set it reaches ROC-AUC 0.930, accuracy 0.842, precision 0.835 and recall 0.892. Recall is the more important clinical metric, and the API returns the raw probability so the decision threshold can be adjusted.
- The champion is then refit on all 918 labelled rows and exported for serving.

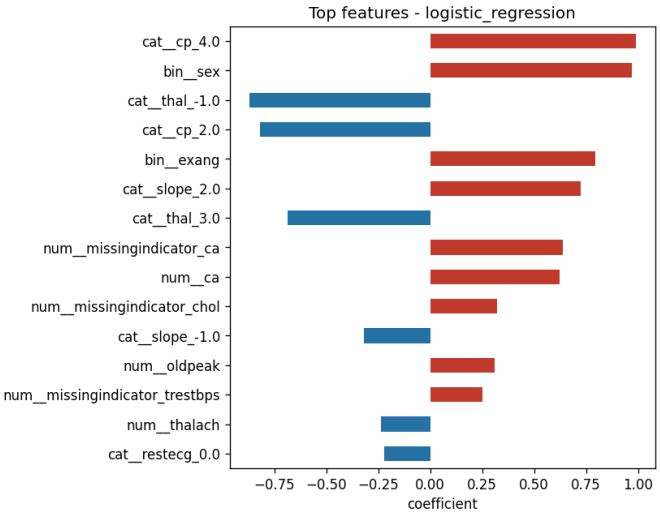


Figure 9. *Largest logistic-regression coefficients (red increases risk).*

5. Experiment tracking (MLflow)

`heart.train` logs every run to MLflow (SQLite backend `mlflow.db`, artifacts in `mlruns/`) under the experiment `heart-disease-classification`, **64 runs** in total:

- **3 parent runs** (one per model family): best parameters, CV mean and standard deviation for 5 metrics, hold-out metrics, fit time, diagnostics and feature-importance plots, the full `cv_results_` CSV, and the sklearn model with an input signature and example.
- **60 nested child runs**, one per grid candidate, with its parameters, CV metrics and rank. These give the full tuning history (notebook 02 plots them).
- **1 champion-final run**: comparison plot and table plus the refit model, registered as `heart-disease-classifier` version 1 with alias **`@champion`**.

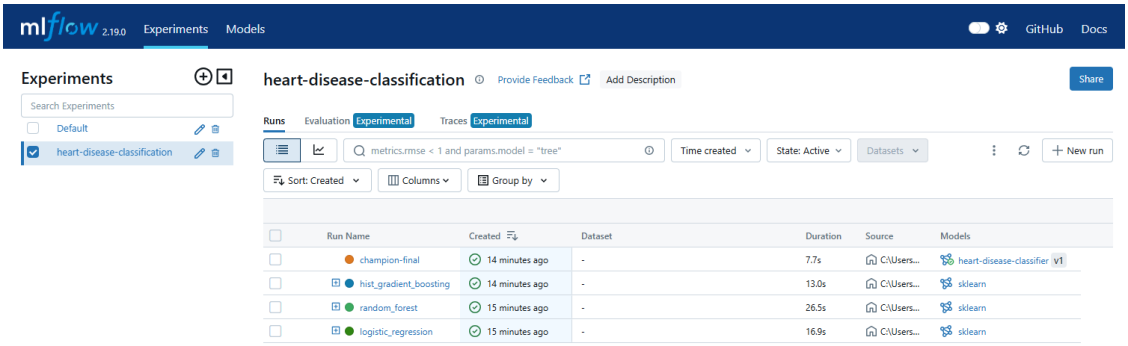


Figure 10. *MLflow experiment: parent runs (expandable to nested candidates) and the registered champion.*

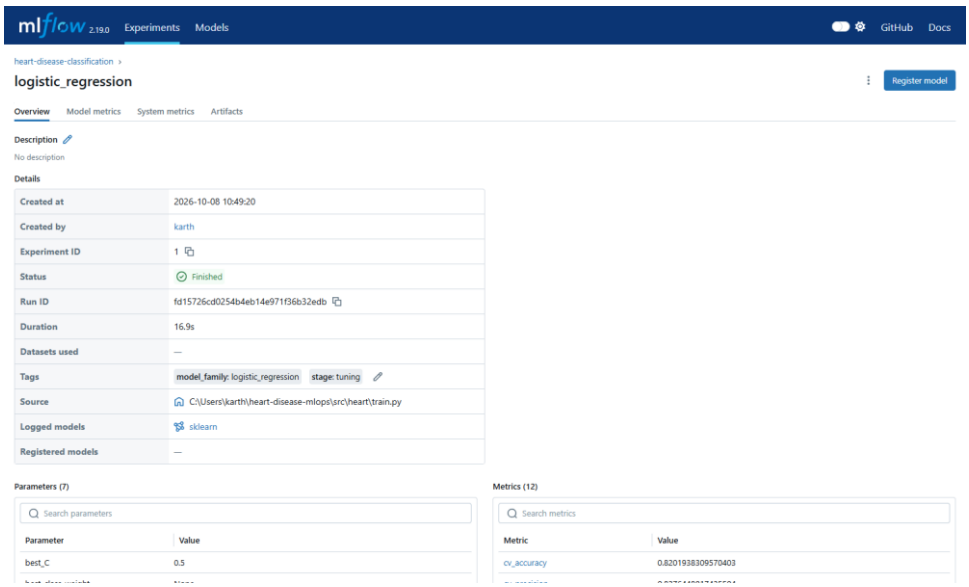


Figure 11. *Logistic-regression parent run: parameters and metrics.*



Figure 12. *Plots logged as run artifacts.*



Figure 13. *Model registry entry with the @champion alias.*

6. Model packaging and reproducibility

Requirement	Implementation
Reusable model format	`models/heart_model.joblib` (complete sklearn Pipeline) plus MLflow `sklearn` flavour in the registry; `models/model_meta.json` stores the run id, parameters, metrics and timestamp

Requirement	Implementation
Environment	pinned `requirements.txt` (training, tests, notebooks) and slim `requirements-serve.txt` (container); installable package defined in `pyproject.toml`
Preprocessing	the `ColumnTransformer` is inside the persisted Pipeline, so a raw JSON request is all that is needed for inference
Determinism	`random_state=42` for splits, CV folds and models; data download is versioned by URL
Inference paths	`python -m heart.predict samples/patient_high_risk.json`, `mlflow.sklearn.load_model('models:/heart-disease-classifier@champion')`, REST API (notebook 03 shows all three agree)

7. CI/CD pipeline and automated testing

7.1 Unit tests

`tests/` contains **34 pytest tests (91 % line coverage)**. They use a synthetic UCI-shaped dataset with injected faults, so they run offline in about a minute:

File	What is verified
test_data.py	target binarisation, impossible zeros and invalid codes become NaN, duplicates removed, schema, no input mutation, '?' parsing
test_features.py	no NaNs after transform, standardisation, missing indicators, fixed one-hot width, all-missing row
test_model.py	learns signal, valid probabilities, metric helper, every model family fits, joblib round-trip, quality bar on the saved artefact
test_api.py	health/ready, /predict contract (prediction + confidence + request id), optional fields, 422 validation, batch, /metrics, 503 without a model
test_training_pipeline.py	full `heart.train` run in a sandbox (MLflow runs + registry alias), EDA writes all figures

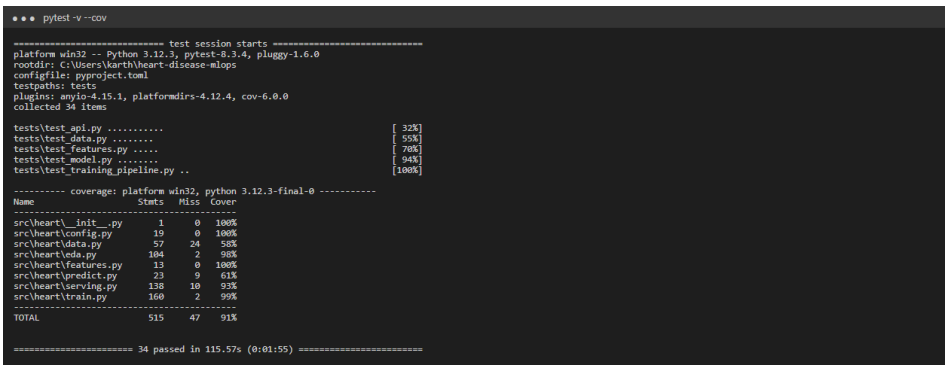


Figure 14. *pytest run with coverage report.*

7.2 GitHub Actions workflow

`.github/workflows/ci.yml` runs on every push and pull request. The jobs are chained with `needs`, so a lint error, failing test, failed quality gate, broken image or failed deployment stops the pipeline with a clear log:

Job	Steps	Artifacts / logging
lint	`ruff check` (GitHub annotations)	inline annotations
test	`pytest` with coverage and JUnit output	`test-reports` (junit.xml, coverage.xml), job summary table
train	download, EDA, train with MLflow, **quality gate CV ROC-AUC >= 0.80**	`model`, `mlflow-tracking`, `reports`; model-comparison table in the summary
docker	build the image with the freshly trained model, run it, smoke test	`docker-image` tarball, container logs
deploy-kind	kind cluster, load image, `deploy_k8s.sh`, smoke test through the Service	`kubectl get all` and pod logs

LAB SCREENSHOT - save as screenshots/50_github_actions_run.png and re-run node scripts/build_report.js

Figure 15. *GitHub Actions run - all five jobs green.*

LAB SCREENSHOT - save as screenshots/51_github_actions_artifacts.png and re-run node scripts/build_report.js

Figure 16. *Workflow run summary and uploaded artifacts.*

8. Model containerisation

The `Dockerfile` builds from `python:3.12-slim` with serving-only dependencies (no MLflow or plotting libraries), installs the `heart` package, bakes in the model artefact, runs as a **non-root user** (uid 10001) and declares a HEALTHCHECK on `/health`. Uvicorn serves `heart.serving:app` on port 8000.

Endpoint	Purpose
POST /predict	one patient as JSON; returns `prediction`, `label`, `confidence`, `probability_disease`, `risk_band`, model name/version
POST /predict/batch	up to 500 patients
GET /health, /ready	liveness (process up) and readiness (model loaded), used by Docker and k8s probes
GET /model-info	metadata and training metrics of the served model
GET /metrics	Prometheus exposition

Inputs are validated by Pydantic with clinical ranges and the allowed category codes. Bad input returns **HTTP 422** with field-level messages, and optional measurements may be omitted.

```
docker build -t heart-disease-api:1.0.0 .
docker run -d -p 8000:8000 --name heart-api heart-disease-api:1.0.0
curl -X POST localhost:8000/predict -H 'Content-Type: application/json' -d
@samples/patient_high_risk.json
{"prediction":1,"label":"heart
disease","confidence":0.9624,"probability_disease":0.9624,"risk_band":"high",...}
```

LAB SCREENSHOT - save as screenshots/40_docker_build_run.png and re-run node scripts/build_report.js

Figure 17. *Docker build, container run and smoke test in the lab (bash scripts/lab_run.sh docker).*



Figure 18. *Interactive OpenAPI docs (/docs): /predict executed with the sample patient.*

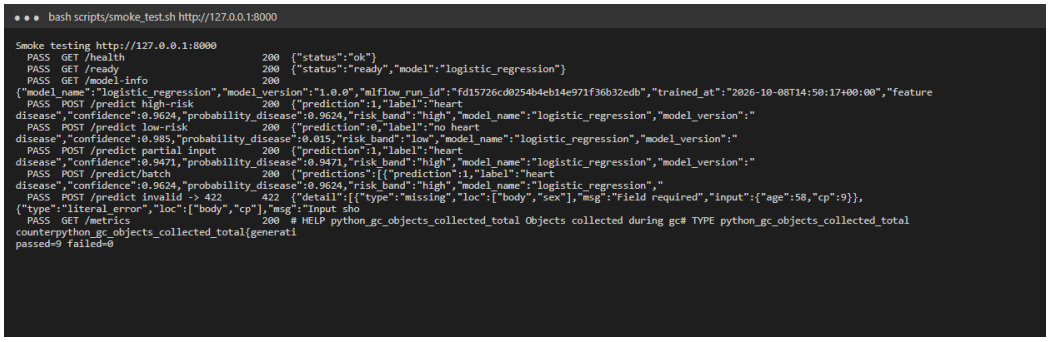


Figure 19. *scripts/smoke_test.sh: 9 checks including validation (422) and metrics.*

9. Production deployment (Kubernetes)

The API is deployed to a local **Minikube** cluster (the same manifests work on Docker Desktop, kind, GKE, EKS or AKS). `scripts/deploy_k8s.sh` applies the kustomization and the monitoring stack, then waits for the rollouts.

Manifest	Highlights
k8s/00-namespace.yaml	isolated namespace `heart-mlops`
k8s/api-deployment.yaml	2 replicas, RollingUpdate with maxUnavailable 0, startup/readiness/liveness probes, CPU/memory requests and limits, non-root, read-only root filesystem, Prometheus scrape annotations
k8s/api-service.yaml	**type LoadBalancer** port 80 to 8000 (EXTERNAL-IP via `minikube tunnel`)
k8s/api-ingress.yaml	**nginx Ingress** host `heart.local`
k8s/api-hpa.yaml	HorizontalPodAutoscaler 2-5 replicas at 70 % CPU
k8s/monitoring/*.yaml	Prometheus (pod discovery, RBAC), Grafana (provisioned datasource and dashboard), NodePorts 30090/30300

```
minikube start --driver=docker && minikube addons enable ingress && minikube addons enable metrics-
server
minikube image load heart-disease-api:1.0.0
bash scripts/deploy_k8s.sh
kubectl -n heart-mlops get all,ingress,hpa
minikube service heart-api -n heart-mlops --url      # or: minikube tunnel / heart.local ingress
bash scripts/smoke_test.sh http://localhost:8080      # via kubectl port-forward svc/heart-api
8080:80
```

LAB SCREENSHOT - save as screenshots/41_k8s_resources.png and re-run node scripts/build_report.js

Figure 20. *kubectl get all,ingress,hpa in namespace heart-mlops.*

LAB SCREENSHOT - save as screenshots/42_k8s_endpoint_verification.png and re-run node scripts/build_report.js

Figure 21. *Endpoint verification through the Kubernetes Service / Ingress.*

Access instructions (local): API `http://localhost:8080/docs` (port-forward), `minikube service heart-api -n heart-mlops --url`, or `http://heart.local` after adding `$(minikube ip) heart.local` to `/etc/hosts`.

10. Monitoring and logging

10.1 Logging

An HTTP middleware gives every request an `x-request-id` and writes one **JSON log line** with method, path, status, latency in ms and client. Predictions are logged with their probability. The logs go to stdout, so they are available through `docker logs` and `kubectl logs` and can be shipped to Loki or ELK without changes.



Figure 22. *Structured request logs (including a rejected 422 request).*

10.2 Metrics, dashboard and alerts

Metric	Type	Use
heart_api_requests_total{method,path,status}	counter	traffic and error rate
heart_api_request_latency_seconds{path}	histogram	p50/p95/p99 latency
heart_api_predictions_total{outcome}	counter	prediction mix (drift signal)

Metric	Type	Use
heart_api_disease_probability	histogram	score distribution
heart_api_model_loaded	gauge	readiness

Prometheus discovers and scrapes every API pod every 10 s. Grafana is provisioned with the *Heart Disease API - Service & Model Monitoring* dashboard. `monitoring/alert_rules.yml` defines four alerts: API down, 5xx rate > 5 %, p95 latency > 500 ms, and a positive-prediction rate above 85 %, which would suggest input drift since the training prevalence is 55 %. `scripts/generate_traffic.py` replays real patients plus 5 % invalid payloads to populate the panels.

LAB SCREENSHOT - save as screenshots/43_prometheus_targets.png and re-run node scripts/build_report.js

Figure 23. *Prometheus targets - each API pod discovered and UP.*

LAB SCREENSHOT - save as screenshots/44_grafana_dashboard.png and re-run node scripts/build_report.js

Figure 24. *Grafana dashboard under generated traffic.*

11. Conclusion and deliverables

The project delivers a heart-disease classifier (CV ROC-AUC 0.888, hold-out ROC-AUC 0.930) behind a validated, observable API. It is reproducible from a clean checkout with one command and automatically linted, tested, trained, quality-gated, containerised and deployed by CI.

Deliverable	Location
Code, Dockerfile, requirements	`src/heart/`, `Dockerfile`, `requirements*.txt`, `pyproject.toml`
Cleaned dataset + download script	`data/processed/heart_clean.csv`, `python -m heart.data`
Notebooks / scripts	`notebooks/01_eda`, `02_model_training`, `03_inference`; `scripts/`
Unit tests	`tests/` (34 tests)
CI/CD workflow	`.github/workflows/ci.yml`
Deployment manifests	`k8s/`, `k8s/monitoring/`, `docker-compose.yml`
Screenshots	`screenshots/`
Repository	https://github.com/<your-account>/heart-disease-mlops
Video	<link to pipeline video recording>

Possible extensions

- Probability calibration and a cost-sensitive threshold chosen with clinicians.
- Feature-level drift detection (e.g. Evidently) on logged requests, with automated retraining through the existing CI train job.
- Push the image to a registry (GHCR) and add a GitOps continuous-delivery step (Argo CD) to a managed cluster.